PHASE 2 / FASTAPI + SQLITE / 01

Phase 2 implementation and verification

Smart Parking System
FastAPI + SQLite Integration

Three ESP32 devices | Twelve slots | Independent MQTT ingestion

Prepared 10 November 2026. This report documents the completed local backend, actual automated test evidence, production observation limits and continuation commands. Times in evidence use UTC; local project timezone is Asia/Calcutta (IST).

Verification class	Actual result
Final automated suite	43 tests; successful=True; failures=0, errors=0, skipped=0.
Real broker/backend/API integration	7 synthetic-fixture checks; passed=True. These are Python publishers, not ESP32 runtime evidence.
Production observation	30 seconds; HTTP health=200; historical observations=0. No new ESP32-to-API delivery pass.
Preservation	Existing databases were not selected for tests or reset. ESP32, circuits, topics, broker configuration, requirements, ML and dashboard remain unchanged.

Phase 1 delivery remains operator-observed, with no original subscriber screenshot or payload archive. The new backend tests do not retroactively establish timestamp accuracy, physical sensing or ESP32 reconnection. Detailed device runtime acceptance remains open.

No credentials, live SQLite files, virtual environment or build output are included in Git. Implementation was completed and tested before the per-file commits; all commits are intended for one final push.

PHASE 2 / FASTAPI + SQLITE / 02

Architecture and data flow

Stage	Responsibility / execution context
ESP32-01 / 02 / 03	P1-P4 / P5-P8 / P9-P12; unchanged firmware publishes four-slot snapshots.
Wokwi private gateway	Firmware broker host stays host.wokwi.internal:1883.
Local Mosquitto	PC-side backend connects to 127.0.0.1:1883. Existing service is unchanged.
MQTT network thread	Connect, subscribe, receive bounded packets, enqueue and service reconnects. No database writes here.
Bounded queue -> persistence worker	Strict validation -> canonical duplicate check -> one SQLite transaction -> manual acknowledgement.
SQLite -> REST API	Short-lived connections per operation. Synchronous endpoints execute in worker threads, independent of ingestion.

Lifecycle and ownership

FastAPI lifespan initializes a new or recognized schema off the event loop, then starts the persistence and MQTT threads. Shutdown signals both threads and joins them; queued or unsuccessfully stored messages remain unacknowledged. Importing backend.main does not create a database or connect to MQTT.

Run a single Uvicorn process and one backend client identity. Multiple workers/reload would create competing subscribers and unnecessary SQLite contention. API handlers do not share SQLite connections with ingestion or perform synchronous database work on the event loop. [1]

Scope

Implemented: validation, history/current persistence, idempotent duplicate handling, reconnect/backpressure behavior, read APIs, health and evidence tooling. Not implemented: ML, predictions, Streamlit, public deployment, API authentication/TLS, lossless historical buffering or new sensor/firmware behavior.

PHASE 2 / FASTAPI + SQLITE / 03

Topics, configuration and telemetry contract

Topic	Device	Slots
smartparking/esp32_01/telemetry	esp32_01	P1-P4
smartparking/esp32_02/telemetry	esp32_02	P5-P8
smartparking/esp32_03/telemetry	esp32_03	P9-P12

Root field	Validation / interpretation
schema_version	Strict integer 1; booleans and coercion rejected.
device_id / slots	Topic must match known device; exactly four distinct assigned slots.
sequence / uptime_ms	Strict unsigned 32-bit integers; reset/wrap is possible.
timestamp / clock_synced	Valid UTC YYYY-MM-DDTHH:MM:SSZ or null; root flag agrees with nullability.
Slot fields	slot_id, distance_cm, occupancy_status, timestamp; no extra fields.
Distance / state	Finite numeric 2-400 cm for OCCUPIED/VACANT; null for UNKNOWN. Device classification preserved.

Payloads are limited to 4,096 bytes. UTF-8, JSON syntax, duplicate object keys, unknown fields, nonfinite numbers, boolean/string numeric substitutions, invalid dates and slot/device mismatches are rejected. Startup null times and mixed null slot times during SNTP acquisition are accepted. Timestamp format validation is not NTP accuracy verification.

Public defaults: PC broker 127.0.0.1:1883; client smartparking-backend; queue 256; stale timeout 30 seconds; MQTT enabled. .env.example now reflects the fixed production topics. Private .env is optional; process environment overrides it. Passwords are excluded from settings repr and logs. No topic-prefix override is exposed in production configuration.

Installed environment: fastapi 0.162.2; pydantic 2.14.3; paho-mqtt 2.2.0; uvicorn 0.54.0; python-dotenv 1.2.4. No project packages were installed or removed.

PHASE 2 / FASTAPI + SQLITE / 04

SQLite schema and preservation policy

Table	Keys and stored information
slots	Twelve fixed slot IDs; assigned device; unique slot/device pair.
telemetry_messages	Integer PK; unique SHA-256 fingerprint; device, sequence, uptime, publish/receipt times, clock flag, inferred session and applied_current.
occupancy_observations	Integer PK; message/slot foreign keys; unique message+slot; state, distance and measurement timestamp.
slot_current	One row per observed slot, pointing to its latest accepted observation.

Every new snapshot inserts a message and all four observations atomically, and updates current pointers only when eligible. Any failure rolls back the entire transaction. History indexes support slot/device/reception-time queries. Foreign keys, state/distance checks, full synchronous writes and bounded lock timeouts are enabled. Reads use independent connections; each connection closes after its operation.

Existing data is never reset

Default live path: %LOCALAPPDATA%\SmartParking\parking.db. Paths must be absolute and outside the checkout and OneDrive. New files are reserved with exclusive creation. Existing files are inspected read-only for application ID 0x53504B32, schema version 1, required tables, exact slot registry and DELETE journal mode. Unrecognized or incomplete files are refused without schema replacement or deletion. There are no automatic migrations or pruning jobs.

The installed SQLite is 3.49.1. Rollback journal mode is intentional: SQLite documents a WAL-reset concurrency bug affecting this version. The project environment was preserved instead of enabling WAL or upgrading Python. Existing journal modes are not silently changed. [3]

Backups and test isolation

Use SQLite's backup API to a new external destination, or stop the backend cleanly before copying. Never copy an open database alone as a claimed consistent backup. Automated tests create only private unique Windows Temp databases and clean up their own fixtures. No default live database was opened by verification. An unrelated test-created database was rejected with an unchanged byte hash.

PHASE 2 / FASTAPI + SQLITE / 05

MQTT reliability, ordering and freshness

Commit-before-acknowledgement

The subscriber uses MQTT 3.1.1, QoS 1, a persistent broker session and manual ACK. The network thread queues bounded data; a separate worker validates and writes. A successful transaction or durable duplicate lookup precedes acknowledgement. This closes the normal callback-return-before-database-write gap. Paho explicitly supports post-processing acknowledgement. [2]

Invalid, retained, unsupported-topic or unsupported-QoS messages are counted and acknowledged without persistence. Database errors hold ACK and retry; health becomes degraded. Queue overflow leaves that packet unacknowledged and requests reconnect. Connection generations prevent a worker acknowledging an old packet ID after a newer connection reuses it. Unexpected worker/network failures are visible; threads do not silently manufacture a pass.

Recovery

The network worker reconnects with 1-30 second backoff and resubscribes after CONNACK. Health requires connection, QoS 1 subscription grants and live threads. Shutdown leaves pending unsuccessful work unacknowledged. Broker session retention, broker persistence, disk availability and Paho process-state limits still apply; there is no end-to-end exactly-once or lossless-crash guarantee.

Duplicate identity and ordering limits

Canonical content includes device, sequence, uptime, times and sorted slot values. Its persistent SHA-256 fingerprint suppresses equivalent duplicates across backend restarts. Duplicate receipt does not refresh current freshness. New observations with older known publication times or older non-boot counters can be history-only. Counter wrap and near-boot low counters infer a new session; equal-counter conflicts remain history-only.

Firmware has no boot ID. Delayed unsynchronized packets, unobserved reboots and identical early-boot snapshots cannot always be distinguished. Session inference is explicitly heuristic. A future boot/session identifier would improve this, but firmware compatibility is preserved in Phase 2.

Current status preserves the last device classification plus observed/stale flags. Stale and unseen slots count as effective UNKNOWN in statistics, never as free spaces. Freshness is based on backend reception time; it cannot prove original measurement age for unsynchronized delayed traffic.

PHASE 2 / FASTAPI + SQLITE / 06

REST API contract and examples

GET endpoint	Behavior
/slots	Twelve slots in numeric order; last state, distance, measured/received times, sequence, stale and observed.
/slots/{slot_id}	One registered slot; 404 for unknown IDs.
/history	Newest observation ID first; items, total, limit and offset. Filters: slot_id, device_id, state, since, until.
/stats	Total 12; fresh occupied/vacant; effective unknown; stale and unseen; occupied/12 percentage.
/health	200 ready or 503 degraded; database_ok plus MQTT connection, subscriptions, queue, counters, timestamps and thread status.
/docs / /openapi.json	Generated interactive documentation / machine-readable API contract.

History time bounds are inclusive and apply to backend reception time. Timezone-aware input is normalized to UTC. limit defaults to 100 and must be 1-1000; offset must be nonnegative. Invalid filters and reversed/naive time ranges return 422. SQLite request errors return a generic 503 without leaking paths. Pagination is consistent within each request, not a frozen multi-request export snapshot.

Actual isolated-test aggregate after restart

```
{
  "total_slots": 12,
  "occupied": 1,
  "vacant": 11,
  "unknown": 0,
  "stale": 0,
  "unseen": 0,
  "occupancy_percent_of_total": 8.33
}
```

These values came from synthetic fixtures on validation topics: P1 was changed to OCCUPIED, the other eleven remained VACANT. They are not live sensor availability. Stale/unseen counts overlap with effective unknown; occupied + vacant + unknown always equals 12.

Health readiness and data freshness are separate: the production observation returned HTTP 200 with twelve unseen slots. MQTT_ENABLED=false explicitly disables ingestion and allows healthy database/API-only operation; health reports enabled=false.

PHASE 2 / FASTAPI + SQLITE / 07

Startup and verification commands

Run from repository root

```
.\.venv\Scripts\python.exe -m uvicorn backend.main:app `
  --host 127.0.0.1 --port 8000 --workers 1

Invoke-RestMethod http://127.0.0.1:8000/slots
Invoke-RestMethod http://127.0.0.1:8000/slots/P1
Invoke-RestMethod "http://127.0.0.1:8000/history?slot_id=P1&limit=20"
Invoke-RestMethod http://127.0.0.1:8000/stats
Invoke-RestMethod http://127.0.0.1:8000/health
```

Use defaults or a private .env copied deliberately from .env.example. Check DATABASE_PATH before startup. Do not use multiple workers, reload mode or a second backend sharing the same MQTT ID. No service configuration change is required.

Executed checks / repeatable commands

```
.\.venv\Scripts\python.exe -m unittest discover -s backend/tests -v

.\.venv\Scripts\python.exe -m backend.tools.verify_integration `
  --output reports/evidence/phase2_broker_followup.json

.\.venv\Scripts\python.exe esp32/tests/test_circuits.py
```

Choose a new evidence output name. Integration starts real local HTTP, a backend MQTT subscriber and a synthetic publisher on a unique smartparking/validation/phase2/... namespace. It uses a private temporary database, tests one backend transport reconnect, restarts the backend and checks durable deduplication. It never publishes production topics or restarts Mosquitto.

Tests use standard-library unittest and urllib, existing Uvicorn and existing Paho; pytest/httpx are not required. External temporary database creation and loopback networking required execution outside the restricted sandbox. This is a tooling permission condition, not an application test pass.

Production observation procedure actually used

A temporary backend used the unchanged production topic map and a unique observer client ID. After all subscriptions were active, a read-only 30-second window collected health, slots, history total and statistics through HTTP. The observer was stopped and only its test session was cleared. No publisher or device simulation was started for that observation.

PHASE 2 / FASTAPI + SQLITE / 08

Actual automated and broker test results

Suite	Tests	Coverage
test_api	12	Real HTTP, OpenAPI, filters, errors, parallel reads, settings, unavailable broker
test_database	12	Atomicity, locks, duplicates, persistence, history, ordering, existing-file preservation
test_ingestion	11	ACK order, invalid/retained packets, queue pressure, retries, callbacks, shutdown
test_validation	8	Three-device contract, slot identities, strict values, dates, malformed JSON

Final run: 43 tests in 3.672 seconds; successful=True; 0 failures, 0 errors, 0 skips. Full test IDs, runner output and source SHA-256 hashes are saved in phase2_unit_tests.json.

Real local-broker check	Result
broker connection and three qos1 subscriptions	PASSED
three devices twelve slots persisted and served	PASSED
duplicate no extra history	PASSED
invalid json contract rejected	PASSED
actual backend transport reconnect and resume	PASSED
synthetic occupancy transition visible	PASSED
backend restart preserves history and deduplication	PASSED

Final integration namespace: smartparking/validation/phase2/688a48fe84. Backend reconnect counter reached 1; stored snapshots=4, duplicate=1 and rejected=1 before restart. History remained 16 observations after restart and replay. This verifies backend reconnection using synthetic data, not ESP32 Wi-Fi recovery.

Failures retained and corrected

Initial sandbox execution could not access Windows Temp. A fixture left a connection open during cleanup; that was fixed and 41 tests passed. The expanded pre-fix 43-test run had 1 failure: unavailable broker exposed a reference to nonexistent paho.mqtt.client.MQTTException. The handler was corrected; the final complete run and real-broker integration were repeated successfully. The failed evidence remains archived.

PHASE 2 / FASTAPI + SQLITE / 09

Production result, limits and troubleshooting

Production observation	Recorded result
UTC start / end	2026-11-10T16:46:31.535Z / 2026-11-10T16:47:02.137Z
MQTT readiness	Connected and subscribed to all three production topics; HTTP health 200.
Received observations	0
Availability result	0 occupied, 0 vacant, 12 UNKNOWN; 12 stale and unseen.
ESP32-to-API acceptance	NOT OBSERVED in this window. Prior Phase 1 delivery remains operator-observed only.

No production messages were available to archive or correlate with actual sensor changes. Therefore live production JSON compatibility, sensor timestamps, periodic arrival cadence and ESP32 recovery are not newly runtime-verified. Static contract checks and synthetic backend tests passed separately.

Symptom	Action
Existing database rejected	Preserve it; choose a new external filename and review ownership/schema. Never delete or reset automatically.
MQTT disconnected / health 503	Check PC broker address/port, credentials, unique backend client ID and service availability.
Health 200, slots UNKNOWN	Check simulations and actual production traffic; distinguish no data from invalid-message rejection.
Database errors / queue grows	Check disk, permissions, locks and duplicate processes. ACK remains withheld; broker retention limits apply.
Stale/history-only readings	Inspect receipt time, device time, counters and restart ambiguity. Do not infer vacancy from silence.
Invalid payloads	Capture exact Serial/subscriber output without secrets and compare with schemas.py; never spoof production topics.

Operational limits: trusted local development only; no API authentication/TLS, retention/pruning automation, schema migration framework, globally reliable boot identity, multi-process ingestion or lossless-outage guarantee. History grows continuously. Plan storage monitoring and deliberate backups. ML/dashboard integration has not started.

PHASE 2 / FASTAPI + SQLITE / 10

Evidence, source files and continuation

Evidence under reports/evidence/	Purpose
phase2_unit_tests.json	Final 43-test run, exact test IDs, output and source hashes
phase2_pre_fix_tests.json	Earlier failed expanded test run before exception-handler fix
phase2_broker_integration.json	Initial seven-check synthetic broker integration pass
phase2_final_integration.json	Final integration rerun after all backend fixes
phase2_production_observation.json	Read-only production observation; zero messages
phase2_repository_checks.json	Circuit regression, protected files, environment and broker hashes

Repository checks: circuit tests exit 0; protected paths changed=[]; requirements match pip freeze=True; broker config hash matches Phase 1=True. Hash comparison normalized hexadecimal case; the initial case-sensitive assertion was corrected without changing the measured bytes.

Implementation map

backend/config.py: settings/path policy; schemas.py: strict telemetry and API models; db.py: storage; mqtt_ingest.py: threads/ACK/reconnect; main.py: lifespan/endpoints; tests/: fixtures and four test modules; tools/verify_integration.py: isolated real-broker verification. backend/__init__.py remains unchanged. Usage and schema instructions are in backend/README.md and database/README.md.

Primary references

[1] FastAPI concurrency and synchronous handlers
https://fastapi.tiangolo.com/async/

[2] Paho MQTT client, manual ACK and persistent sessions
https://eclipse.dev/paho/files/paho.mqtt.python/html/client.html

[3] SQLite WAL-reset bug and affected versions
https://sqlite.org/wal.html#the_wal_reset_bug

Continuation: run the single-worker backend with a reviewed external database path, start the three existing Wokwi simulations manually, and archive fresh production messages plus API results and sensor-change correlations. Test controlled device recovery separately. Keep operator observations and independently saved evidence distinct. No ML or dashboard work is authorized by this report.

Reproduce with reports/generate_phase2_report.py using ReportLab and Windows Arial/Consolas fonts. PDF tooling comes from the bundled environment; it does not modify the project's dependency snapshot. The PDF was rendered and visually checked before delivery.